Digital Signal Processing Exercise Sheet 3

LTI Systems

Script: Chapter 3 (Linear Time-Invariant Systems)

Learning goals

- Compute convolutions by hand (graphically and in tables) and in C; verify the algebraic properties of convolution numerically.
- Derive impulse and step responses of the two workhorse smoothers – moving average (FIR) and exponential smoother (IIR) – from their difference equations.
- Quantify the noise-versus-lag trade-off of a smoother (noise gain, mean delay, ramp lag) and relate it to the step response.
- Apply LTI filters to the per-tooth engine speed in real time and understand why the missing-tooth gap must be handled *before* the filter (time invariance in the tooth domain).

Setup: C programs on the simulated ESP32

Exercises 3.2 and 3.3 a) are plain C programs without any circuit: they compute, print tables and draw ASCII plots. They run in Wokwi on an ESP32 that is used as a small C computer – no second environment is needed.

1. Open the project folder `sheet03_conv` (Exercise 3.2) or `sheet03_rpmoffline` (Exercise 3.3 a) in `code/wokwi_vscode/` in VS Code (or open the workspace `sheet03.code-workspace` there, which contains all projects of this sheet). The program is in `src/sketch.ino`; the headers it needs (`asciiplot.h`, `engine_signals.h`) are copies in `src/`.
2. Complete the `TODO`s, build (*PlatformIO: Build*) and start the simulator (`F1` → *Wokwi: Start Simulator*). The program runs once in `setup()` and writes everything to the Serial Monitor; the last line is `done.` Copy the results from there.
3. Why the ESP32 and not the Uno: the programs need tens of kilobytes of RAM (the Uno has 2 KB), and on the Uno `double` is only a 32-bit `float` – the rounding errors of Exercise 3.2 b) would be those of single precision. The ESP32 computes `double` in software: correct, but slow, so Exercise 3.3 a) needs a while in the simulator.
4. The first ESP32 build downloads the toolchain, about 1 GB (see the README in `code/wokwi_vscode/`). In the browser the same projects are in `code/wokwi/`: create an ESP32 project and add the header files as extra tabs.

Background

A discrete-time LTI system is completely described by its impulse response $h[n] = S\{\delta[n]\}$; the output for any input is the convolution

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k]\, h[n-k].$$

Two smoothers are used throughout the course (written for the per-tooth speed $r[k]$, but valid for any sequence):

$$\text{MA (FIR):}\quad y[n] = \frac{1}{N}\sum_{k=0}^{N-1} x[n-k] = y[n-1] + \frac{x[n]-x[n-N]}{N},$$

$$\text{EMA (IIR):}\quad y[n] = a\,y[n-1] + (1-a)\,x[n] = y[n-1] + \alpha\,(x[n]-y[n-1]), \qquad \alpha = 1-a,$$

called moving average and exponential smoother (exponential moving average). Useful characteristics of a smoother with $\sum_n h[n] = 1$: the *noise gain* $\sum_n h^2[n]$ (variance reduction for white noise) and the *mean delay* $D = \sum_n n\,h[n]$.

Exercise 3.1 Convolution and smoothers by hand

PAPER

a) Compute $y = x * h$ for $x[n] = \{\underline{1}, 2, 0, -1\}$ and $h[n] = \{\underline{1}, -1, 0.5\}$ (underlined: $n = 0$) twice: graphically (flip and shift h) and with a table of products $x[k]h[n-k]$. Check the result with $\sum_n y[n] = \sum_n x[n] \cdot \sum_n h[n]$ and explain why this identity holds.

b) Determine impulse response $h[n]$ and step response $s[n]$ of the MA of length N and of the EMA (from the difference equation by iteration and from $s[n] = \sum_{k\le n} h[k]$). After how many samples does each reach 50 % and 90 % of a step for $N = 16$ and $a = 15/17$?

c) Compute noise gain and mean delay D of both smoothers. Show that for a ramp input $x[n] = c\,n$ every smoother with $\sum h = 1$ has the steady-state output $y[n] = c\,(n - D)$, and that $D = \sum_{n\ge 0}(1 - s[n])$, i.e. the ramp lag can be read off the step response. Which a gives the EMA the same noise gain as the MA of length N? Compare the mean delays.

d) Show that $x[n] = \mathrm{e}^{\mathrm{j}\Omega n}$ is an eigenfunction of every LTI system, $y[n] = H(\Omega)\,x[n]$ with $H(\Omega) = \sum_k h[k]\mathrm{e}^{-\mathrm{j}\Omega k}$. Evaluate $|H|$ for MA_{16}, MA_{30} and the EMA with $a = 15/17$ at $\Omega = 2\pi/30$. (On the 60-tooth wheel, sampled once per 6°, this is the 2nd engine order – the speed ripple caused by the combustions.)

Exercise 3.2 Convolution engine

WOKWI

Program: project `sheet03_conv` (ESP32 without circuit, see Setup).

a) Implement `conv(x, nx, h, nh, y)` (full linear convolution, output length $n_x + n_h - 1$, no out-of-range accesses) and check it with the example of Exercise 3.1 a).

b) Verify commutativity, associativity and distributivity with random sequences of lengths 200, 50 and 30. Why are the deviations not exactly zero? Why might they be exactly zero for "nice" test numbers?

c) Implement the exponential smoother recursively (`exp_smooth()`) and compare it with the convolution by its impulse response truncated to L samples ($a = 0.9$, $L = 10 \ldots 160$). Relate the error to a^L and compare the number of multiplications per output sample. Verify the step response formula of Exercise 3.1 b).

d) Convolve the impulse response of an 8-point MA with itself. Explain the shape and the mean delay of the result and compare the step responses of one MA and of the cascade.

Exercise 3.3 Smoothing the RPM estimate

WOKWI

The per-tooth speed $r[k] = 1/T_k$ of Sheet 1 contains the real speed fluctuation (2nd order, ±1 %) and timer quantisation. We smooth it with an MA and an EMA in the *tooth domain* (one sample per tooth). Three ways to treat the gap (period $\approx 3T$) are compared: `GAP_MODE` 0 – use $1/T_{\mathrm{gap}}$ like a normal value; 1 – correct it to $3/T_{\mathrm{gap}}$ (one sample); 2 – insert $3/T_{\mathrm{gap}}$ three times (one sample per 6°).

a) *Offline prediction* (ESP32 without circuit, see Setup; project `sheet03_rpmoffline`). The program simulates the crank edges of the virtual engine with 0.5 µs time stamps. Implement the gap handling, `moving_average()` (running sum) and `exp_smoother()`. Report: at constant 3000 rpm mean, standard deviation and min/max of raw, MA_{16}, MA_{30}, $EMA_{2/17}$ for all three gap modes; for a step 3000→5000 rpm the 50 %/90 % times in samples for $N = 8, 16, 32$ and $\alpha = 2/(N+1)$; for a ramp of 3000 rpm/s the lag error in rpm, in samples and in ms. Compare with Exercise 3.1.

b) *Real time* (WOKWI, `code/wokwi/sheet03_rpmfilter/`). The ISR pushes tooth periods into a FIFO; `loop()` computes and filters the speed. Implement the same three steps in the sketch. It prints one line per revolution and, whenever the raw speed jumps by more than 150 rpm between two teeth, a capture block of 128 samples (32 before the jump). Start at 3000 rpm and *click* the speed slider at 5000 rpm. Determine the 50 %/90 % points of both filters from the capture and the ramp lag $D = \sum(1 - s[n])$. (Dragging the slider produces a staircase of steps, not a ramp – why is the step response sufficient anyway?)

c) Repeat with `GAP_MODE` 0 and 1. Explain the per-revolution values and the dips in the capture. Why does the gap violate the time invariance of the tooth-domain processing, and why is only mode 2 compatible with the exact ripple cancellation of MA_{30}?

d) Design: the injection timing needs the speed during an acceleration of 5000 rpm/s at 1000 rpm with an error below 0.5 %, while the 2nd-order ripple should be suppressed. Is this possible with an MA in the tooth domain? What do real ECUs do (keyword: *segment time*)?

Deliverables

3.1: convolution table, derivations of h, s, noise gain, D, $|H|$. 3.2: program and output, explanation of the rounding and truncation errors, plot of the triangular impulse response. 3.3: completed `sheet03_rpmoffline` sketch with result tables; sketch; capture of a step for all three gap modes (plot in any tool, e.g. a spreadsheet), 50/90 % points and ramp lag; answer to d).